Report for Deep Learning Assignment 2
By Team_30

Name	Roll No
Pallab Biswas	**23EE10092**
Arghyadeep Ghosh	**25CS91R03**
Piyush Sarangi	**23EE10082**
Ajitesh Jamulkar	**22CS10004**
Ansh Sahu	**22CS30010**

Our Approach

Architecture (Tiny Recursive Model – TRM)

We use a Tiny Recursive Model (TRM) that applies a shared Transformer module recursively over the input for 6 outer reasoning cycles, each containing 3 inner Transformer layers, with weights shared across cycles.

Justification: Weight sharing across recursive cycles drastically reduces parameter count while enabling iterative refinement, which is critical for solving multi-step ARC tasks under a strict 50M budget.

Data Augmentation (8x)

Each ARC task is represented as a sequence of tokens encoding grid cell values, with multiple input-output examples ,we generated 7 more pairs for each training task by flipping , mirroring etc.

Justification: 400 tasks are very less to learn from. More data can often help to generalize just we have to be careful that it does not overfit. Since arc tasks are abstract, there is nearly zero chance that more data will be bad.

Positional Encoding (3D RoPE)

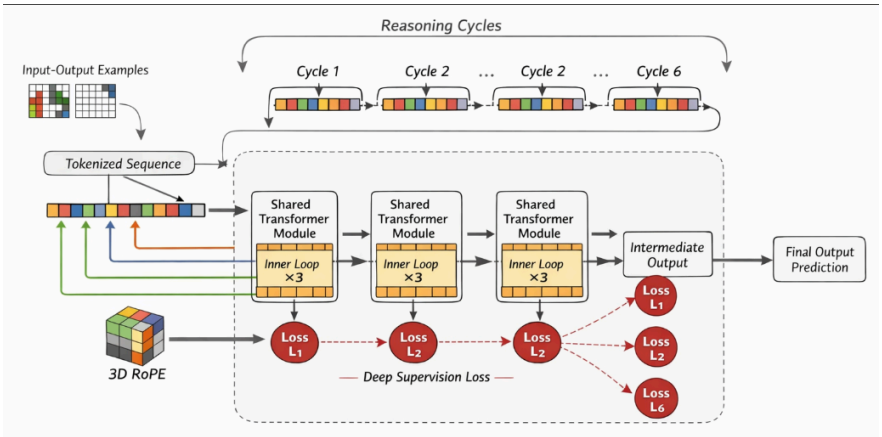
We use 3D Rotary Positional Encoding (RoPE) applied across row, column, and example (pair) dimensions, allowing the model to encode spatial structure within grids and relationships across multiple demonstrations.

Justification: 3D RoPE enables the model to generalize spatial transformations while also aligning corresponding positions across different input-output pairs.

Loss Function (Deep Supervision)

We employ deep supervision, applying cross-entropy loss at the output of each outer recursive cycle, with losses averaged (or weighted) across all cycles.

Justification: Supervising intermediate recursive steps encourages the model to produce progressively refined predictions and improves gradient flow through repeated applications of the shared module.



Reasoning Depth (Outer = 3, Inner = 6)

We use 3 outer recursive cycles, each consisting of 6 inner Transformer layers, where the shared TRM module is applied iteratively to refine predictions.

Justification: A deeper inner loop (6 layers) increases per-step reasoning capacity, while a smaller number of outer cycles (3) enables iterative refinement without excessive computational overhead, striking a balance between expressivity and efficiency under the 50M constraint.

Ablation Study

Our Model is **TRM** using **3D Rope** with **deep supervision loss** and **reasoning depth of 3 outer and 6 inner cycles.**

Base line choices will be

- Replace 3DRope with 2DRope

Choice	Eval Acc	Diff	Why
3D Rope (ours)	1/40	+0.25%	Using **2D RoPE instead of 3D RoPE** removes the encoding of the **example/pair dimension**, so the model can no longer properly align corresponding positions across different input-output demonstrations.
2D Rope (baseline)	0/40	-0.25%	

- Replace Deep Supervision Loss with CE Loss

Choice	Eval Acc	Diff	Why
Deep Supervision Loss(ours)	1/40	+0.25%	Using **only final cross-entropy (CE) loss** removes supervision from intermediate reasoning steps, so the model is trained only on the last prediction rather than the full reasoning trajectory.
CE Loss (baseline)	0/40	-0.25%	

- Reduce Reasoning Depth from 6*3 to 1*1

Choice	Eval Acc	Diff	Why
Reasoning Depth 3 outer and 6 inner (ours)	**1/40**	+0.25%	ARC tasks often require sequential reasoning (e.g., detect → transform → place), and a shallow 1×1 setup lacks both sufficient capacity (inner depth) and iterative refinement (outer cycles), leading to incomplete or incorrect rule execution.
Reasoning Depth 1 Outer and 1 inner (baseline)	**0/40**	-0.25%	

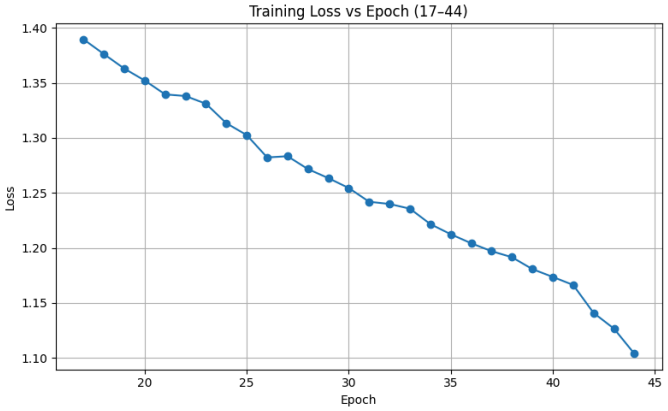
Results

Our model's parameter count is within the **50 million limit**.

```
============================================================
  Original TRM (3-D RoPE, deep-sup, 6×3)
============================================================
  token_emb                          1,152
  pair_emb                           1,632
  proj_in                           27,744
  f_inner                          173,504
  proj_ya                           18,528
  f_update                         173,504
  head                                970
  halt_head                            97
  drop                                  0
------------------------------------------------------------
  TOTAL                            397,131
  ✓ Within 50M budget  (0.3971M / 50.000M)
============================================================
```

We were able to **solve 1 evaluation task** completely in 2 attempts

The loss vs epoch graph is shown below



We got **Evaluation Accuracy as 1/400**

```
print(f"  Accuracy: {final_acc*100:.2f}%")
print("="*55)
```

```
[Eval] Loading best checkpoint…
[Eval] Loading evaluation tasks…

=======================================================
  FINAL EVALUATION — ARC-AGI-1 (400 eval tasks)
=======================================================
  Solved : 1 / 400
  Accuracy: 0.25%
=======================================================
```

What did not work

We evaluated several alternative architectural choices that we hypothesized would be competitive for ARC-style reasoning, but empirically observed consistent degradation in performance relative to the proposed TRM-based approach.

First, we implemented a **Transformer encoder-only architecture**, where concatenated input–output demonstrations and the query grid were processed using bidirectional self-attention, with predictions obtained via masked token classification. The underlying assumption was that full-context attention would suffice for inferring transformation rules. However, this model exhibited unstable and often incomplete predictions. We hypothesize that the absence of an explicit generative or iterative mechanism limits the model's ability to map latent representations to structured outputs, particularly for tasks requiring sequential composition of transformations.

Next, we considered a standard **encoder–decoder Transformer**, aiming to decouple representation learning and generation. While this formulation improved output consistency compared to the encoder-only variant, it still underperformed significantly. We attribute this to the inherently **single-pass nature** of encoder–decoder architectures, where reasoning is compressed into a fixed-depth computation graph. ARC tasks frequently require multi-step hypothesis refinement (e.g., object identification followed by rule application and spatial rearrangement), which cannot be effectively captured without iterative updates or recurrence.

We further explored a **CNN–Transformer hybrid model**, wherein convolutional layers were used to encode spatial features prior to Transformer-based reasoning. Despite the intuitive appeal of incorporating spatial inductive biases, this approach did not yield improvements. A likely explanation is that CNNs enforce locality and translation equivariance, which may be misaligned with the **non-local and symbolic transformations** characteristic of ARC tasks. Additionally, the inclusion of convolutional components reduces the effective parameter allocation for the reasoning module under a fixed budget, thereby limiting representational capacity where it is most needed.

Overall, these negative results suggest that **single-pass architectures and strong locality-biased feature extractors are insufficient for ARC**, and reinforce the importance of **iterative, recursive reasoning with shared parameters**, as realized in the TRM formulation.